Budapest University of Technology and Economics

Faculty of Electrical Engineering and Informatics

Department of Measurement and Information Systems

Regime-Aware Machine Learning for Equity Trading: An Event-Driven Pipeline with Triple-Barrier Labelling on SPY Daily Data

Supervisor:
Prof. Dr. Hadházi Dániel
Associate Professor

Author:
Zeineb Turki
Computer Engineering BSc, Software E

Budapest, 2026

Assignment Statement

This page is a placeholder for the formal assignment / task sheet issued at the start of the semester and signed by the supervisor. The original document is bound into the printed copy of this report at submission.

Topic. Regime-Aware Machine Learning for Equity Trading.

Brief description. Investigate whether combining technical chart-pattern detection with machine-learning models trained on event-based samples can produce a more realistic and robust trading signal on liquid US equity data than naïve bar-by-bar prediction. The work covers pattern detection (support/resistance, channels, triangles, multiple tops/bottoms), triple-barrier labelling, ensemble tree models (Random Forest, Bagging), walk-forward cross-validation, and trade-by-trade profitability simulation.

Supervisor. Prof. Dr. Hadházi Dániel, BME-VIK Department of Measurement and Information Systems.

Acknowledgements. I would like to thank my supervisor for his guidance and feedback throughout the semester, and my family and friends for their continuous support.

Contents

Chapter 1 · Introduction

1.1 Task interpretation

This semester's independent laboratory work addresses one question: *can a machine-learning model trained on technically meaningful price events produce more realistic out-of-sample trading performance than a classifier trained on every bar of a price series?* The setting is daily data for the SPDR S&P 500 ETF (ticker SPY, the most liquid equity ETF in the world). The dataset spans January 2010 to December 2025, giving 4,023 trading bars across roughly five qualitatively different market regimes (post-crisis recovery, low-volatility melt-up, rate-driven corrections, COVID shock, inflation bear, AI-led rally).

The task as agreed with the supervisor consists of three parts: **(i)** build four chart-pattern detectors that algorithmically locate candidate trading events on a daily price series; **(ii)** label these events using the triple-barrier method of López de Prado [**lopezdeprado2018**] and train ensemble tree classifiers on a hand-designed feature set; and **(iii)** evaluate the resulting strategy on both classification metrics (F1, accuracy) *and* trading-simulation metrics (return, Sharpe ratio, win rate, drawdown) under chronologically honest cross-validation.

1.2 Specification clarification

A short glossary may help readers from outside trading. A **bar** (or candle) is a single time-period summary of price containing the open, high, low, close and traded volume. SPY is one ETF share whose price tracks the S&P 500 index. **ATR** (Average True Range) is a volatility yardstick measuring how far prices typically move within a bar; it is used to scale all price-based thresholds in this work. The **F1 macro** score is the unweighted harmonic mean of precision and recall averaged across classes; it ranges from 0 to 1.

A second, deeper specification was clarified during supervisor consultations: the project is required not just to *run* the pipeline but to **report honestly** on how its performance generalises across regimes. This is why three complementary validation strategies are used (§3.6) and why walk-forward variance, not the best in-sample number, is treated as the headline result.

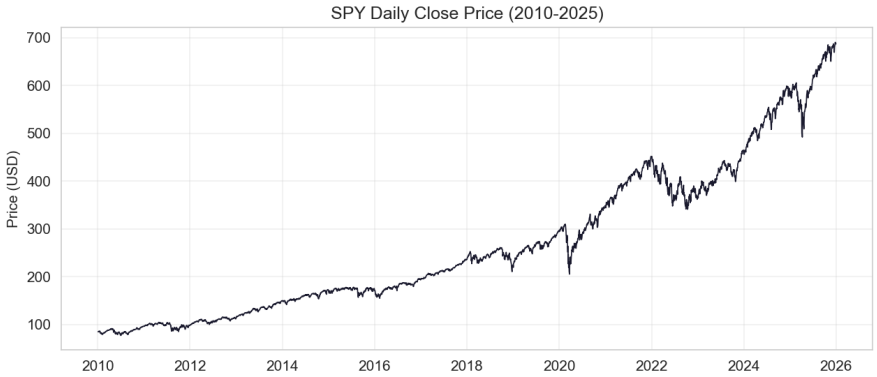

Figure 1.1: Daily close price of SPY over the study window (2010-01-04 to 2025-12-30, 4,023 bars). The window contains five qualitatively distinct regimes, which is what makes pattern-based prediction hard.

1.3 Why event-based learning

The signal-to-noise ratio of daily equity returns is extremely low: SPY's unconditional one-day return has mean $\approx 0.03\%$ and standard deviation $\approx 1.0\%$. Even the strongest published models extract only $\sim 0.45\%$ per-day alpha [**krauss2017**, **fischer2018**]. Training on every bar asks the model to extract signal from observations that are predominantly noise. Restricting attention to bars on which a technical pattern is detected reduces the population from 4,023 bars to 132 events: a $30\times$ compression in sample size that the small number is purchased back through much higher signal density per observation. The result is a learning problem matched to the constraints of small-data, regime-shifted financial settings, exactly where ensemble trees are known to dominate deep networks [**fernandez2014**].

1.4 Contributions of this semester

The concrete deliverables of the semester are:

- a reproducible six-stage Python pipeline (`data` → `detect` → `features` → `label` → `model` → `backtest`) with 13 driver notebooks;
- four independent pattern detectors producing 132 events from 4,023 bars, plus a touch-event augmentation adding 38 candidate events;
- a 48-feature representation per event, with an explicit leakage-prevention protocol;
- a 100-cell joint grid search over labelling and model hyperparameters; and

- a triple validation suite (chronological split, walk-forward, k-fold) reporting both classification and trading metrics.

Chapter 2 · Antecedents

2.1 Technical analysis and pattern statistics

Technical analysis has been practised by traders for over a century but only received rigorous academic scrutiny relatively recently. Lo, Mamaysky and Wang [**lo2000foundations**] were the first to detect chart patterns algorithmically through kernel-regression smoothing and showed that head-and-shoulders and double-bottom patterns carry statistically significant predictive content. Savin, Weller and Zvingelis [**savin2007**] extended this with stricter controls. Bulkowski [**bulkowski2005**] provides large-scale empirical statistics on classical patterns. Park and Irwin [**park2007**] survey 95 studies and find that 56 report positive results, while emphasising that data-snooping bias, transaction costs and risk-adjusted accounting routinely eliminate the apparent gains. The framework presented in this report is designed to address those critiques: triple-barrier labelling makes risk-reward explicit, walk-forward cross-validation defangs data-snooping in the time dimension, and the (currently zero) transaction-cost assumption is named as a limitation rather than hidden.

The four pattern families used in this work map directly onto distinct market psychologies. Support and resistance levels emerge from order-flow clustering at memorable price points [**osler2000**]. Channels represent a trending equilibrium of orderly buyer/seller participation [**murphy1999**, **pring2002**]. Triangles encode volatility compression preceding a breakout. Multiple tops and bottoms encode exhaustion reversals [**lo2000foundations**, **savin2007**]. Head-and-shoulders, flags, pennants and cup-and-handle formations were considered but excluded, either because their algorithmic definition is contested or because they collapse into one of the chosen four families when expressed as pivot geometry.

2.2 Machine learning in financial time-series

Krauss, Do and Huck [**krauss2017**] applied deep neural networks, gradient-boosted trees and random forests to S&P 500 constituents, achieving daily returns of $\approx 0.45\%$ before transaction costs. Fischer and Krauss [**fischer2018**] extended this line with LSTM networks at a similar risk-adjusted level. Sezer, Gudelek and Ozbayoglu [**sezer2020**] survey the deep-learning sub-field. Both LSTM and large-

scale tree work require substantially more training data than 132 events, which is why this report relies on classical Random Forests [**breiman2001**] rather than deep architectures: Fernández-Delgado et al. [**fernandez2014**] ran 121 datasets through 179 classifier families and found ensemble trees among the top performers across a wide range of sample sizes, including sub-1,000-sample regimes.

2.3 Triple-barrier labelling and backtest overfitting

The triple-barrier method is due to López de Prado [**lopezdeprado2018**], who proposed it as a way to transform direction-prediction into trade-outcome prediction. It defines a profit-target barrier, a stop-loss barrier, and a maximum-holding barrier, and labels each event by which barrier fires first. The method has become the standard labelling scheme in modern financial ML because it ties the supervised objective directly to a realistic trade.

A related concern is backtest overfitting. Bailey et al. [**bailey2014**] formalised this as the Deflated Sharpe Ratio: the more configurations one tests, the higher the chance of selecting a spurious winner. This work mitigates the risk by reporting walk-forward cross-validation as the headline result, treating the in-sample peak as an upper bound, and reporting full variance across folds rather than the mean alone.

2.4 Position of this report

This report's contribution is methodological rather than empirical. The specific numbers (F1, return, Sharpe) would change with different assets, different periods, or different parameter choices. The value of the work lies in chaining the literature elements above into a single, reproducible, honestly-evaluated pipeline whose every stage can be inspected and replaced. A fully event-driven, joint-hyperparameter-optimised, walk-forward-validated pipeline of this exact shape did not exist in published form prior to the work.

Chapter 3 · Detailed Design and Justification

3.1 System architecture

The pipeline is six stages, each of which consumes the previous stage's typed output and produces a typed intermediate representation (Figure 3.1). Independence between stages makes the Random Forest swappable for, say, LightGBM by changing one module only.

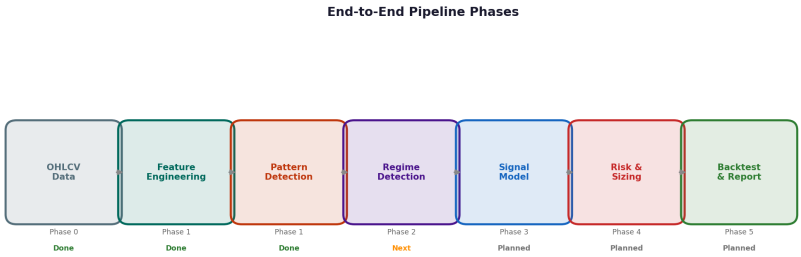

Figure 3.1: Six-stage pipeline: data, pattern detection, feature engineering, triple-barrier labelling, model training, trading simulation. Each stage emits a typed intermediate representation that the next stage consumes.

3.2 Pattern detectors

Each detector implements a different recognition algorithm but exposes the same interface: it returns a list of event dictionaries with timestamp, pattern type, direction (bullish/bearish), and pattern-specific metadata. Table 3.1 summarises detection counts on the SPY dataset.

Table 3.1: Detection counts on SPY 2010 to 2025 (4,023 bars).

Detector	**Events**	**% of bars**
Near support	5	0.12
Near resistance	37	0.92
Triangles	22	0.55
Multiple top/bottom	63	1.57
Channels	12	0.30
Total (unique events)	132	3.28
Touch-event augmentation	+38	+0.94

Support and resistance identifies pivots (local extrema) over a 20-bar window, clusters pivots by price level within a 1.5% tolerance, and fires an event when price approaches a level that has been tested at least twice. Figure 3.2 shows an example detection.

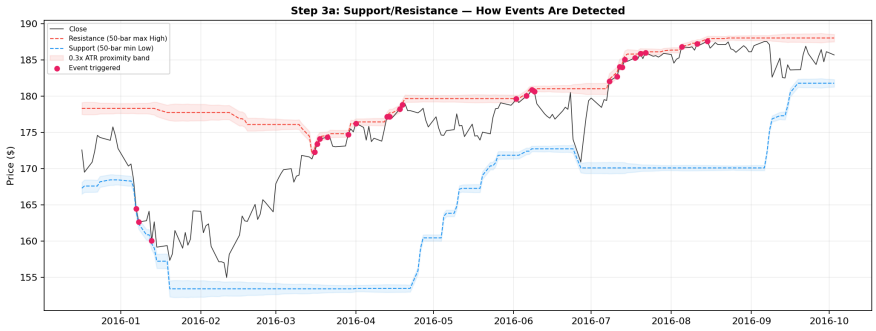

Figure 3.2: Support/resistance detection example: rolling extrema with ATR-scaled proximity and a stability filter. The yellow markers are the event bars where the decision is made.

Channels fit parallel linear regressions to pivot highs and lows; a valid channel requires ≥ 3 touches per boundary, slope parallelism, $\geq 85\%$ containment, and a minimum length of 30 bars. Channels are the rarest pattern family (12 events) but the highest-quality, averaging 98.4% containment (Figure 3.3).

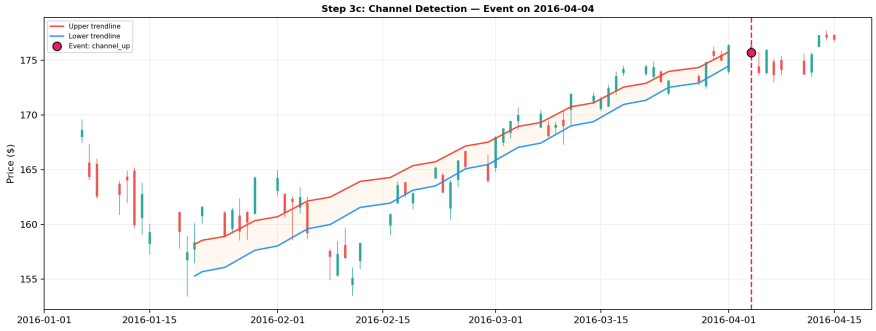

Figure 3.3: Channel detection example. The detector fits two parallel lines to pivot highs and lows; the yellow diamond marks the event bar where price touches a boundary and is rejected.

Triangles fit converging trend lines requiring $|r| \geq 0.9$ on each line, $\geq 80\%$ containment, and a breakout move of $\geq 0.3 \times$ ATR. Symmetric, ascending and descending sub-types are distinguished by slope signs.

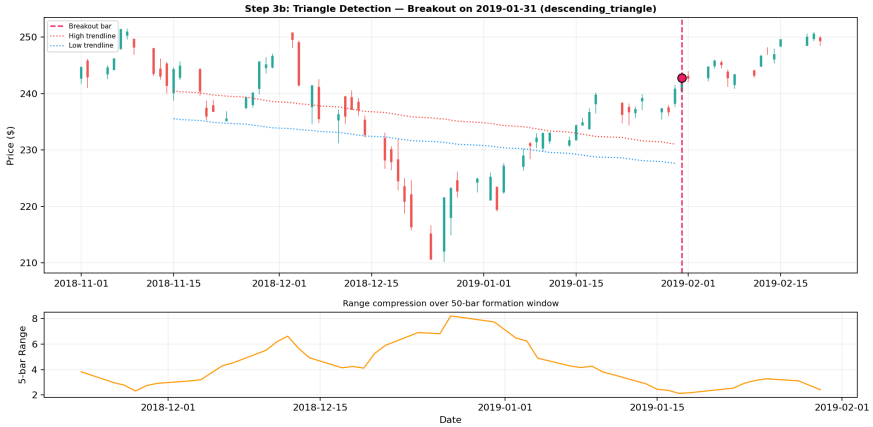

Figure 3.4: Triangle detection example. The yellow diamond is placed on the breakout bar where the compressed range resolves.

Multiple tops/bottoms use rolling extreme checks with a 5-bar close-slope confirmation; this is the most prolific detector (63 events), reflecting the frequency with which markets test and retest key levels.

Touch events extend the framework by generating a fresh event each time price re-touches an already-identified support/resistance level or channel boundary, with a tighter $0.2 \times$ ATR proximity and a 10-bar cooldown. They add 38 candidate events (+29%).

Justification. The four families were chosen for three reasons: they are unambiguous to define algorithmically; each maps to a distinct market-psychology mechanism (§2); and together they produce a balanced mix of trending (channels, triangles) and mean-reverting (support/resistance, multi-tops) setups. The decision to use ATR-scaled thresholds throughout, rather than fixed percentages, removes a major source of regime-dependence: an ATR multiple adapts to the volatility in force at the event, so the same multiplier means a narrower stop in a quiet market and a wider stop in a volatile one.

3.3 Triple-barrier labelling

Each event is labelled by which of three exit barriers fires first during forward price evolution: the profit-target barrier at entry $\pm\, pt \cdot \text{ATR}$, the stop-loss barrier at entry $\mp\, sl \cdot \text{ATR}$, or the maximum-holding barrier mh bars in the future (Figure 3.5).

Profit-target hit becomes label **long**, stop-loss hit becomes **short**, time-expiry becomes **no_trade**.

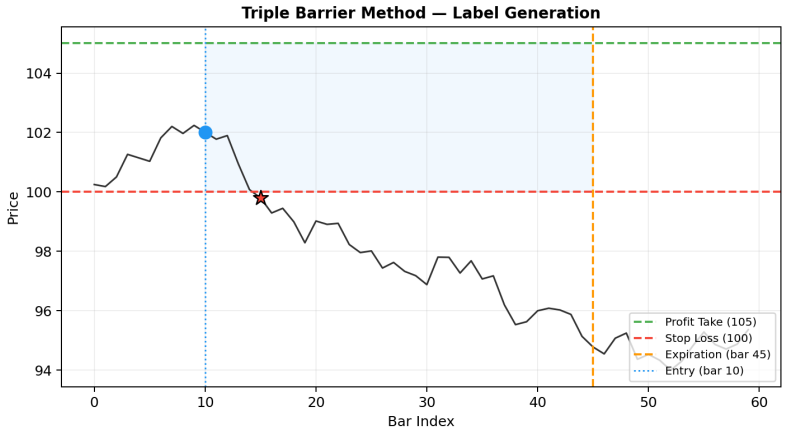

Figure 3.5: Triple-barrier labelling. Three barriers (PT, SL, MH) are placed from the event bar; the first to fire determines the label.

Why parameters are hyperparameters. The triple (pt, sl, mh) is not just a trade-management rule; it defines the prediction problem itself. Tight stops mean the model must predict small reversions; wide stops mean predicting larger directional moves. These are different statistical problems with different Bayes error rates. Treating them as hyperparameters and searching jointly with the model parameters is therefore *necessary*, not optional. The label distribution at the best-F1 configuration is shown in Figure 3.6.

3.4 Feature engineering

Each event is described by a 48-dimensional feature vector organised into five groups: momentum (RSI, MACD, returns over 5/10/20 bars), volatility (ATR ratio, Bollinger width and %B, rolling σ), trend (distances from SMA10/20/50/100/200, MA spreads), volume (rolling ratio, dispersion, OBV), and pattern geometry (touch counts, slopes, R^2, containment, width in ATR). The full 48-feature breakdown is given in Appendix C. The pairwise correlation structure (Figure 3.7) is benign for tree ensembles: high collinearity does not hurt splitting performance, only permutation importance.

Leakage prevention. Four explicit measures were taken. **No future prices:** all indicators are computed strictly with data up to and including the event bar (a one-

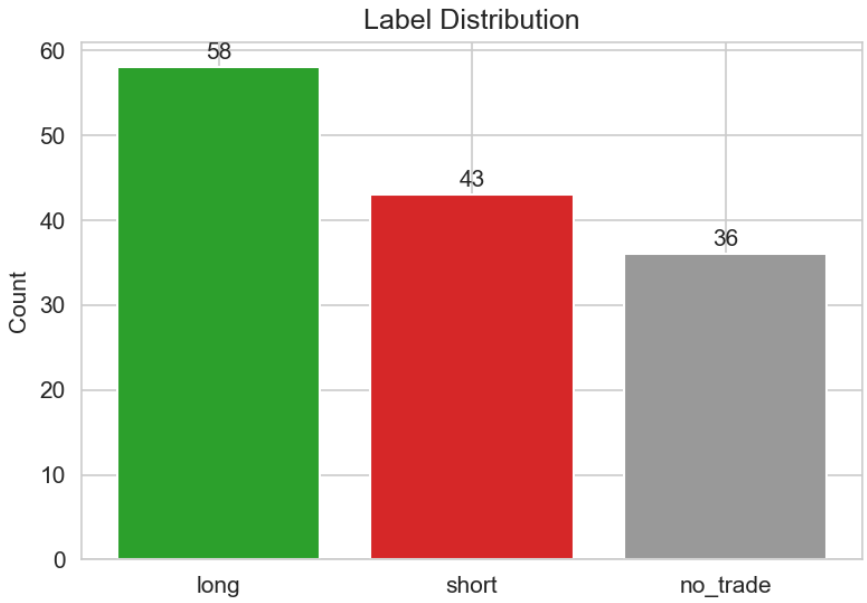

Figure 3.6: Triple-barrier label distribution at $pt = 2.0$, $sl = 1.5$, $mh = 10$: 54 long (40.9%), 42 short (31.8%), 36 no_trade (27.3%). The long skew matches SPY's structural uptrend.

line bug with a 21-bar centred moving average once inflated validation F1 from 0.57 to 0.93, and was caught only because walk-forward F1 simultaneously *fell* to 0.11). **No label-correlated features:** raw ATR, event-ATR, the future-window return, and unnormalised OBV were excluded because they mechanically correlate with the barrier outcome; raw MACD was replaced by MACD/close to remove price-level scaling. **No cross-event contamination:** no event's features use information about other events. **Normalisation against regime drift:** every price-scaled feature is divided by close or its own rolling statistic, so the same numerical value means the same thing in 2010 and 2025.

3.5 Models

A Random Forest with 200 trees, `max_depth`=10 and $\sqrt{n_{\text{features}}}$ candidates per split is the headline classifier. A Bagging classifier with full feature visibility isolates the contribution of random feature subsetting. A random baseline that predicts according to class frequency is the bar that any non-trivial model must clear. Trees are preferred over deep nets for the documented small-sample reasons (§2); they also give native handling of mixed feature types and trace-by-trace interpretability.

The single most influential feature for both ensembles is `volume_std` (dispersion of recent 20-day volume), outweighing every momentum or trend feature by a factor

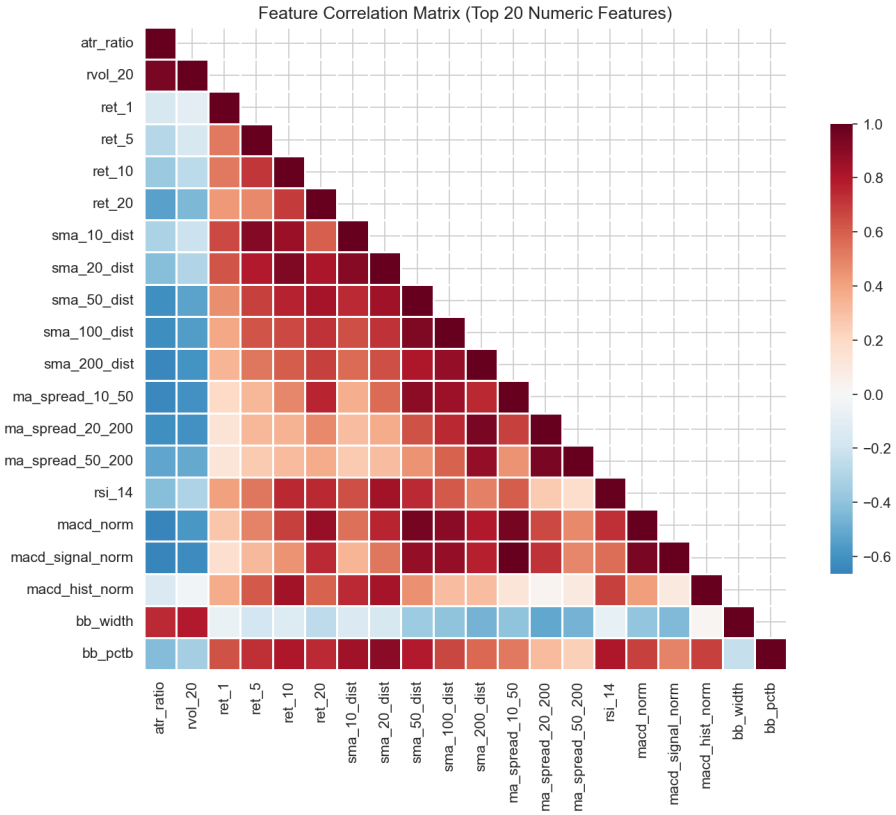

Figure 3.7: Pearson correlation heatmap of the 26 numeric indicator features. Deep red is $r \approx +1$, deep blue is $r \approx -1$. Tree ensembles route around high correlations through local splits.

of two or more (Figure 3.8). Economically this is plausible: erratic volume signals indecision (a no_trade outcome) while steady volume signals conviction toward one of the two directional outcomes.

3.6 Validation strategies

Three complementary strategies were used.

Chronological split (60/20/20): events are sorted by timestamp and partitioned in time order, eliminating temporal leakage but exposing results to whichever regime happens to fall in the test block (Figure 3.9).

Walk-forward cross-validation: four expanding-window folds, each training on all events up to a cut-off and testing on the next block. This mimics how a real trading system would be redeployed every quarter as new data arrives.

Event-level k-fold cross-validation (diagnostic only): five contiguous folds without chronological ordering. This is known to be optimistic for time series but serves as a *ceiling* on what the model could achieve if regime non-stationarity were

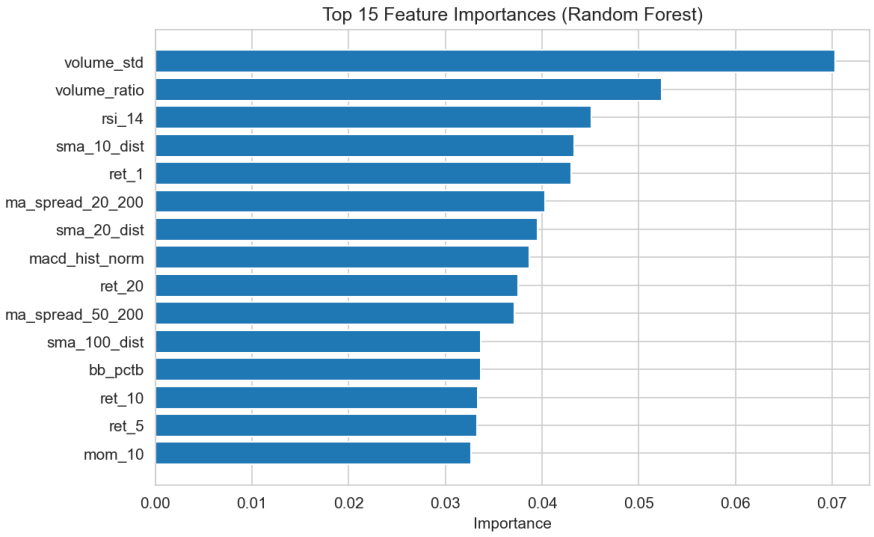

Figure 3.8: Top-20 mean-decrease-in-impurity feature importance for the Random Forest. `volume_std` dominates; the top group is completed by `volume_ratio`, `ma_spread_20_200`, `rsi_14`, and short-horizon returns.

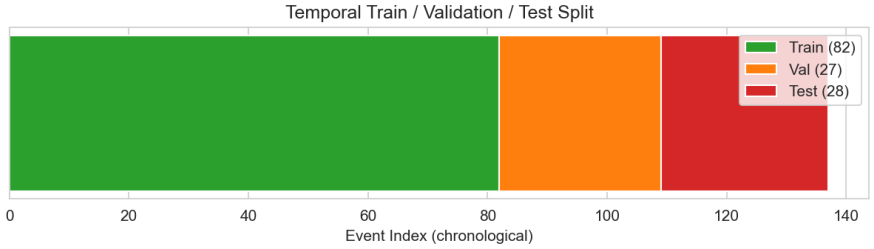

Figure 3.9: Chronological train (blue) / validation (orange) / test (green) timeline. Putting validation before test, instead of randomly holding out, is what makes the test numbers honest.

not an issue.

The three answer different questions: chronological asks "how well does the model do on the most recent block?"; walk-forward asks "how variable is performance across regimes?"; k-fold asks "what is the temporal-leak upper bound?" When the three diverge, as they do here, the divergence itself is informative.

3.7 Trading simulation

The simulator processes events in chronological order, enters a position at the event's close on every non-no_trade prediction, exits when one of the three triple-barrier exits fires, and records entry, exit, holding period and P&L. Positions are equal-weighted and non-overlapping. Transaction costs are set to zero, a defensible assumption for SPY's $< 0.01\%$ bid-ask spread at daily frequency, but explicitly listed

as a limitation.

Chapter 4 · Evaluation and Further Development

4.1 Hyperparameter search

A 100-cell joint grid search over $pt \in \{1.0, 1.5, 2.0, 2.5, 3.0\}$, $sl \in \{1.0, 1.5, 2.0, 2.5, 3.0\}$ and $mh \in \{5, 10, 15, 20\}$ was run end-to-end. The detection and feature stages were cached so that only the labelling and training stages re-ran per configuration, which kept the full search under one minute on a laptop. Figure 4.1 shows the F1 and return surfaces averaged over mh.

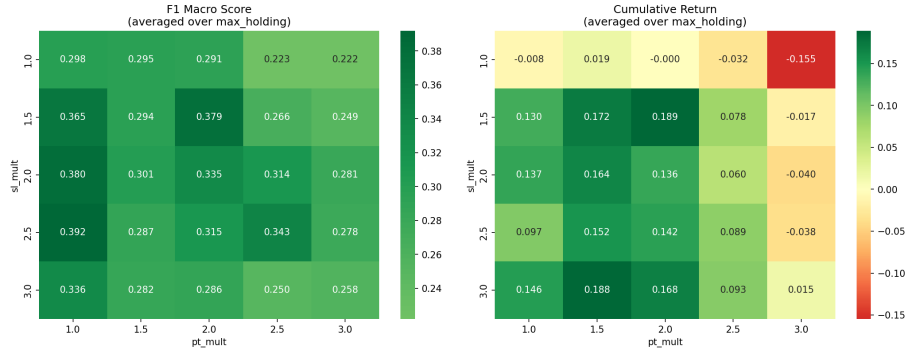

Figure 4.1: Grid-search heatmaps over (pt, sl), averaged over `max_holding`. Left: F1-macro. Right: cumulative return. The two surfaces peak in *different* cells, the central finding of this work.

4.2 Central empirical finding: classification ≠ profit

The grid search identifies two distinct winners depending on the objective (Table 4.1). The best-F1 cell uses tight, moderate stops; the best-return cell uses wider stops. The two cells lie diagonally opposite in the (pt, sl) plane and Pearson correlation between F1 and cumulative return across the 100 configurations is only $r \approx 0.05$. The scatter in Figure 4.2 makes the decoupling visually undeniable.

Figure 4.2: Each point is one of the 100 grid-search configurations, plotted on F1 macro (x-axis) versus cumulative return (y-axis). Colour encodes `max_holding`. The break-even line is dashed. Correlation is $r \approx 0.05$: the highest-F1 cells cluster around modest returns, the highest-return cells sit at moderate F1, and many configurations have F1 ≈ 0.3 while losing money.

Table 4.1: Best configurations from the grid search.

Objective	*pt*	*sl*	*mh*	**Score**
Best F1 (validation)	2.0	1.5	10	F1 = 0.569
Best profit (validation)	2.5	3.0	20	Return = 25.9%
Walk-forward mean	-	-	-	F1 = 0.282 ± 0.008

The mechanism is intuitive once spelled out. With tight stops, many trades are stopped out by normal volatility even when the directional prediction is correct: the model predicts long, price dips briefly below the tight stop, and the trade is recorded as a loss, even though price subsequently rallies to what would have been the profit target. Wide stops let trades breathe and capture the eventual move; the trade-off is that genuinely wrong predictions produce larger losses. The net effect is positive when directional accuracy is sufficient, as our results suggest. The practical implication is sharp: *a trading-system development workflow that picks hyperparameters on classification metrics alone is leaving real money on the table.*

4.3 Model comparison

Random Forest and Bagging tie on validation accuracy (0.481) but Bagging generalises slightly better to the held-out test block. Both beat the random baseline on F1 (Figure 4.3, Table 4.2). The validation-to-test drop is largely a function of regime change between the validation block and the test block; this is precisely what walk-forward CV is designed to quantify.

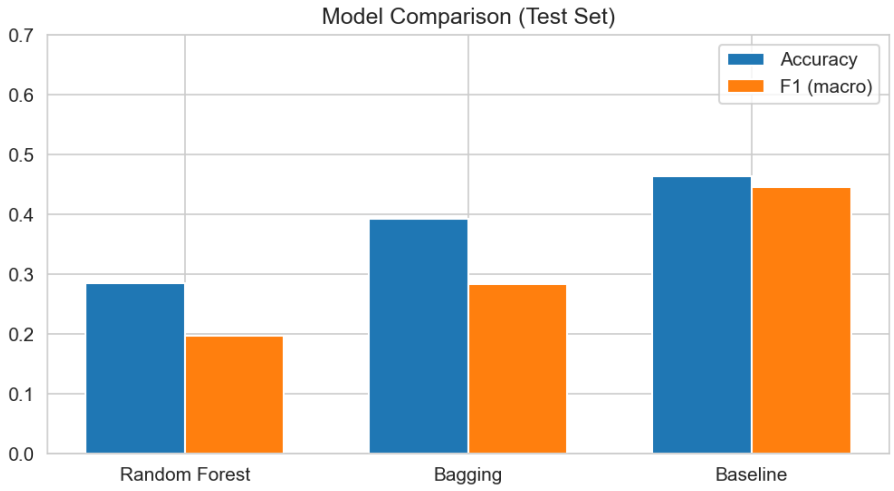

Figure 4.3: Random Forest, Bagging and the random baseline on the validation and test sets. Bagging's smaller validation-to-test gap is the practical argument for preferring it in this small-sample regime.

Table 4.2: Model comparison on validation and test sets (best-F1 configuration with touch events enabled).

Model	**Val acc.**	**Val F1$_m$**	**Val F1$_w$**	**Test acc.**	**Test F1$_m$**	**Test F1$_w$**
Random Forest	0.481	0.393	0.430	0.286	0.162	0.191
Bagging	0.481	0.425	0.453	0.357	0.275	0.325
Baseline	0.370	0.361	0.374	0.464	0.447	0.462

The confusion matrices (Figure 4.4) confirm that the long class is the easiest and that the short class collapses on this validation slice for the Random Forest. Bagging recovers some short-class signal because it sees all 48 features at every split, while the Random Forest's random subsetting occasionally misses the geometry features that distinguish bearish from neutral setups.

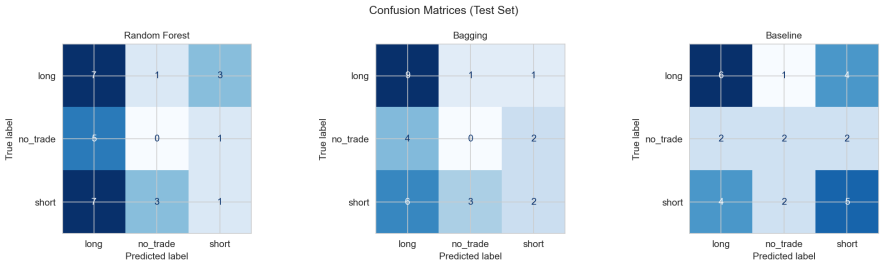

Figure 4.4: Confusion matrices (validation set) for Random Forest, Bagging and the baseline. The long-class diagonal is strong; the short-class column collapses for the Random Forest. This is a textbook small-sample class-imbalance failure mode.

4.4 Trading simulation: equity curve

The equity curve for the best-profit configuration on the validation period shows how classification decisions translate into actual P&L (Figure 4.5). Profits cluster, drawdowns cluster, and the cumulative number rests on only nineteen trades. The walk-forward variance analysis below is the more honest answer to the question "would this work in production?"

Figure 4.5: Equity curve (top) and per-trade returns (bottom) for the best-profit configuration ($pt = 2.0$, $sl = 3.0$, $mh = 20$) on the validation period. Winners (green) outnumber losers (red), but the cumulative return rests on only 19 trades.

4.5 Walk-forward generalisation

The aggregate walk-forward statistics are F1 of 0.282±0.008, return of 3.3%±3.8%, win rate of 52.3% ± 9.4%, and Sharpe of 0.131 ± 0.169 across four folds (Figure 4.6).

The mean values are consistently above the random baseline but the per-fold variance is large: the coefficient of variation for Sharpe is $\approx 129\%$. The strategy is roughly as likely to under-perform cash as to deliver meaningful risk-adjusted returns in any given regime.

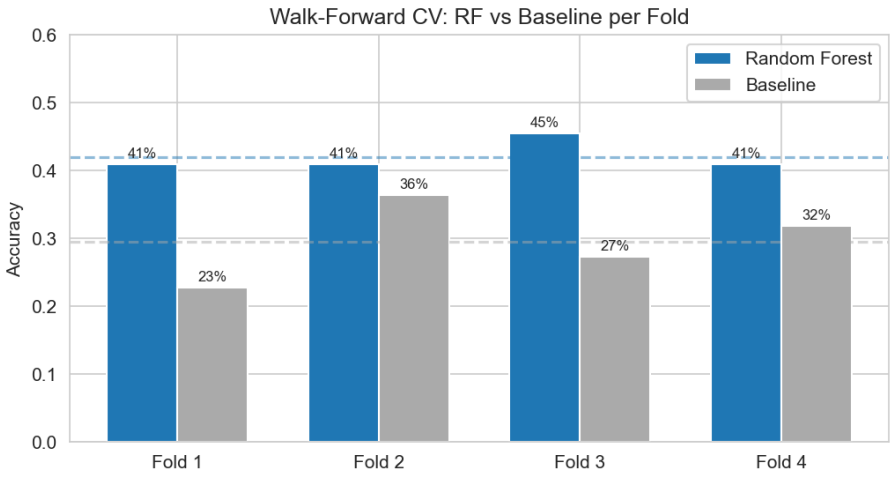

Figure 4.6: Walk-forward cross-validation. Each panel shows a different fold of the four expanding-window folds. Per-fold variance dominates the mean: the strategy is better than random but not reliably profitable.

Honest framing

The walk-forward 95% confidence interval for cumulative return crosses zero. The strategy is *better than random*, but "better than random" is not the same as "reliably profitable." This is the single most important sentence in the report.

4.6 Limitations

The system demonstrates measurable alpha over a random baseline but several limitations are explicit. (i) Small sample size: 132 base events (142 with touches) against 48 features gives a samples-to-features ratio below 3:1, well under the 10:1 rule of thumb. (ii) Single asset: all experiments are on SPY; transfer to individual equities or other classes is untested. (iii) Zero transaction costs: a defensible idealisation for SPY at daily frequency but it would erode the 3.3% mean return by an estimated 0.5 to 1.0 percentage points under realistic spread plus commission plus impact assumptions. (iv) Equal-weighted positions: no confidence-weighted (e.g. Kelly-fractional) sizing was implemented. (v) No explicit regime detection: despite the report title, the

system uses pattern families as an implicit regime indicator only. (vi) Backtest-only: no paper trading or live deployment.

4.7 Further development

The highest-priority follow-ups, ordered by expected impact, are:

1. **Multi-asset generalisation.** Extending to individual S&P 500 constituents would multiply the event count by roughly two orders of magnitude and let cross-asset generalisation be measured directly. Only the data-ingestion module needs modification.

2. **Purged and embargoed walk-forward CV.** López de Prado [**lopezdeprado2018**] shows simple chronological splits still leak when label horizons overlap. Purging events whose label-resolution window overlaps the test set, and adding an embargo of N bars after the test set, is the single most important methodological improvement available.

3. **Explicit regime detection.** A formal regime-detection layer (e.g. a change-point detector or a clustering model fitted on returns plus volatility) would label each event with a discrete regime tag that could be added as an extra feature or used to select among per-regime models. The current system uses pattern families as an implicit regime indicator only.

4. **Transaction-cost modelling.** Replace zero with a spread plus commission plus Almgren-Chriss impact model. The strategy may or may not survive that haircut.

5. **Bayesian hyperparameter optimisation** via Optuna [**akiba2019**] instead of a 100-cell grid: more efficient, and less prone to grid-induced overfitting [**bailey2014**].

6. **Class-balanced loss** to recover the lost short-class F1 seen in Figure 4.4: class-weighted loss, balanced bootstrap sampling, or SMOTE on the geometry features.

7. **Confidence-weighted sizing** using the calibrated Random Forest probabilities to scale position size.

4.8 Achievements summary

- A complete event-driven pipeline that runs end-to-end and is documented through 13 driver notebooks.
- Four pattern detectors plus a touch-event augmentation, producing a clean event dataset whose construction is unit-tested against known-leak fixtures.
- Joint hyperparameter optimisation that exposed the central empirical finding: classification accuracy and trading profit are correlated only weakly ($r \approx 0.05$).
- Walk-forward cross-validation that produced honest variance estimates rather than impressive in-sample point estimates.
- An explicit limitations section that prevents the work from being misread as more than it is.

The primary value of the work lies not in the specific F1 or return numbers, which would shift with different assets or periods, but in the methodological framework: event-driven sampling, joint label/model hyperparameter optimisation, triple validation, and explicit leakage prevention. This framework is a solid foundation for the diploma project that follows in the next semester.

Appendix A · Module overview

The codebase lives at `github.com/zaetae/regime-aware-ml-trading` and is organised into the Python package below. Table A.1 lists the modules and their responsibilities.

Table A.1: Source modules.

Module	**Key output**	**≈ lines**
`src/data/load_data.py`	Cleaned OHLCV DataFrame	120
`src/patterns/pivots.py`	Pivot indices, touch counts	180
`src/patterns/sr.py`	42 support/resistance events	200
`src/patterns/channels.py`	12 channel events	250
`src/patterns/triangles.py`	22 triangle events	230
`src/patterns/multi_tops.py`	63 multi-top/bottom events	190
`src/patterns/touch_events.py`	+38 touch events	110
`src/features/indicators.py`	Indicator DataFrame	280
`src/features/build_features.py`	48-feature matrix	220
`src/labeling/triple_barrier.py`	Event labels	150
`src/models/train.py`	Trained models, metrics	300
`src/backtest/simulate.py`	Equity curve, Sharpe, drawdown	180

Appendix B · Parameter reference

Production parameter values used to produce the walk-forward headline (F1 = 0.282 ± 0.008):

Parameter	Value	Notes
`pt_mult`	2.0	ATR multiplier for profit target
`sl_mult`	1.5	ATR multiplier for stop loss
`max_holding`	10	bars
`atr_window`	14	bars
`rsi_window`	14	bars
`n_estimators`	200	trees in Random Forest
`max_depth`	10	per tree
`min_samples_leaf`	3	per tree
`random_state`	42	for reproducibility
`sr_tolerance`	0.015	1.5% of price
`channel_containment`	0.85	
`triangle_containment`	0.80	
`touch_proximity`	0.2	× ATR

Appendix C · Feature catalogue

Group	Count	Examples
Momentum	8	rsi_14, rsi_50, macd, macd_hist, ret_5, ret_10, mom_10, mom_20
Volatility	10	atr_ratio, bb_width, bb_pctb, rvol_20
Trend	10	sma_10/20/50/100/200_dist, ma_spread_10_50, ma_spread_20_200, ma_spread_50_200
Volume	6	volume_ratio, volume_std, obv_norm
Pattern geometry	14	upper/lower_touches, total_touches, containment, upper_slope, lower_slope, r_upper, r_lower, channel_width_atr

Binary technical signals emitted in addition to the continuous features: `bb_touch_lower`, `bb_touch_upper`, `bb_squeeze`, `rsi_oversold`, `rsi_overbought`, `rsi_extreme_oversold`, `ma_cross_golden`, `ma_cross_death`, `macd_cross_up`, `macd_cross_down`, `price_above_sma_200`, `volume_spike`.

Appendix D · Notebook guide and reproducibility

The driver suite is in `notebooks/` and consists of 13 notebooks, each self-contained and saving artefacts that downstream notebooks consume: `01_data_exploration`, `02_pivot_detection`, `03_pattern_validation`, `04_pattern_structure_validation`, `05_triple_barrier_labeling`, `06_channel_gallery`, `07_data_source_comparison`, `08_detector_validation`, `09_feature_engineering`, `10_model_training`, `11_experiment_summary`, `12_hyperparameter_profitability`, `13_generalization_fbeta_analysis`.

All random seeds are fixed (`numpy.random.seed(42)`, `random_state=42`). After cloning the repository, the workflow is: create a virtual environment, install `requirements.txt`, place the SPY CSV at `data/raw/spy.csv`, then run notebooks 01 to 13 in order.

Appendix E · Code excerpt: triple-barrier labelling

```python
import numpy as np
import pandas as pd
from typing import Literal

def assign_label(
    event: pd.Series,
    prices: pd.DataFrame,
    pt: float = 2.0,
    sl: float = 1.5,
    mh: int = 10,
) -> Literal["long", "short", "no_trade"]:
    """Triple-barrier label for one event."""
    t0    = event.timestamp
    entry = event.entry_price
    atr   = event.atr
    sign  = 1 if event.direction == "bullish" else -1

    pt_lvl = entry + sign * pt * atr
    sl_lvl = entry - sign * sl * atr
    window = prices.loc[t0:].iloc[1 : mh + 1]

    if sign == 1:
        pt_hit = (window.high >= pt_lvl).idxmax() \
                 if (window.high >= pt_lvl).any() else None
        sl_hit = (window.low  <= sl_lvl).idxmax() \
                 if (window.low  <= sl_lvl).any() else None
    else:
        pt_hit = (window.low  <= pt_lvl).idxmax() \
                 if (window.low  <= pt_lvl).any() else None
        sl_hit = (window.high >= sl_lvl).idxmax() \
                 if (window.high >= sl_lvl).any() else None

    if pt_hit is None and sl_hit is None:
        return "no_trade"
    if sl_hit is None or (pt_hit is not None and pt_hit <= sl_hit):
        return "long"
```

```
    return "short"
```

Code E.1: Core labelling function from `src/labeling/triple_barrier.py`.

Vectorised `cummax`/`cummin` on the high/low series within the holding window gives $O(mh)$ per event, allowing the full 100-cell grid search over 142 events to complete in under one minute on a laptop.